

HTR Academy — Personalized Learning

Complete Technical & Operational Documentation

Version: 2.0 **Last Updated:** March 2026 **Classification:** Internal — Engineering, Product, Operations **Revenue Tier:** Subscriber-Only Feature

Table of Contents

1.

[Feature Overview](#)
2.

[User Guide](#)
3.

[Architecture Overview](#)
4.

[Data Flow — End to End](#)
5.

[Backend — Deep Technical Reference](#)
6.

[Frontend — Deep Technical Reference](#)
7.

[The Sanity CMS Content Library](#)
8.

[Persistence Layer — localStorage](#)
9.

[Authentication & Access Control](#)
10.

[Environment Variables & Configuration](#)
11.

[Maintenance & Operations](#)
12.

[Scaling & Upgrade Paths](#)
13.

[Troubleshooting Guide](#)
14.

[Known Limitations & Roadmap](#)
15.

[Security Considerations](#)

1. Feature Overview

Personalized Learning is an AI-powered curriculum generation system embedded in the HTR Academy. It is the platform's primary **subscriber-exclusive** feature and a core revenue driver.

Given a learner's role, experience level, topic selections, learning goals, available time, and preferred difficulty, it generates a bespoke multi-week learning path containing:

- **Reading items** — 100–130 word AI-generated lessons, tailored to the learner's role and difficulty

•

Case study items — AI-generated narrative analysis of real US/Vermont healthcare organizations, linked (when a confident match exists) to a full published case study in the HTR Academy with a personalized bridge paragraph explaining its specific relevance

•

Knowledge checks — 3 scenario-based multiple-choice questions with explanations, calibrated to the learner's role

•

Recommended resources — supplementary links to HTR courses, external CMS/CMMI pages, and other materials

Every path is unique. Two subscribers with the same topics but different roles — say, a clinician and a hospital CFO — receive structurally and conceptually different curricula: different framing, different examples, different quiz scenarios.

What Makes It Personalized (Precisely)

The AI receives the following inputs and uses all of them when generating every sentence:

Input	How It Shapes the Output
Role	Frames every concept through the lens of that role's responsibilities (e.g., a clinician gets clinical outcomes language; a CFO gets benchmark and margin language)
Experience years	Sets the assumed baseline knowledge — a 10+ year veteran does not receive definitions of basic concepts

Input	How It Shapes the Output
Topics (up to 5)	Determines the subject matter of every week's reading and case study
Difficulty	Controls depth — Foundational means first principles; Advanced means leading-edge strategy and nuance
Time per week	Directly controls how many items are generated per week (target ~28 min/item)
Timeline weeks	Controls the total length — 1 to 12 weeks
Goals	Shapes emphasis within topics — two learners studying VBC but with different goals (policy fluency vs. financial analysis) receive materially different content
Custom goal	Free-text goal that is appended verbatim to the goals section of the AI prompt
Format preference	Signals the AI toward certain content styles (deep readings vs. interactive quizzes vs. case studies)

2. User Guide

2.1 Accessing Personalized Learning

Navigate to **HTR Academy → Personalized Learning** tab, or directly to:

```
/academy?tab=personalized
```

This tab is visible to all users but **path generation requires an active subscriber account**. Non-subscribers see the wizard but receive a 403 error when they attempt to generate.

2.2 The 5-Step Wizard

Step 1 — Who Are You? (Role & Experience)

Select your professional role from 9 options:

Role ID	Label
clinician	Clinician / Provider
administrator	Hospital Administrator
policy_maker	Policy Maker
technology_leader	Technology Leader
finance	Finance / Revenue Cycle
researcher	Researcher / Analyst
consultant	Consultant / Advisor
student	Student / Early Career
other	Other

Then select your years of healthcare experience: 0–2, 3–5, 6–10, or 10+.

Why it matters: Role is the single most influential input. The AI has explicit framing instructions for each role (e.g., a **technology_leader** receives content framed around "digital health strategy, interoperability, and technology

ROI"). A clinician learning about value-based care gets clinical workflow and HEDIS metric language. A CFO learning about the same topic gets benchmark calculations, shared savings distributions, and operating margin analysis.

Step 2 — What Do You Want to Learn? (Focus Areas)

Select 1–5 topics from 10 available options. Topics map to HTR's five content pillars:

Topic ID	Label	Pillar
health-economics	Health Economics & Value-Based Care	Economics
payment-reform	Payment Reform & APMs	Economics
clinical-innovation	Clinical Innovation & Quality	Clinical
population-health	Population Health Management	Clinical
health-policy	Health Policy & Reform	Policy
regulatory	Regulatory Compliance & Risk	Policy
health-technology	Healthcare Technology & Digital Health	Technology
data-analytics	Data Analytics & AI in Healthcare	Technology
health-equity	Health Equity & SDOH	Equity
leadership	Healthcare Leadership & Strategy	Leadership

Limit of 5 topics is enforced in both the UI (disables unselected options after 5 chosen) and the backend validator (`v[:8]` hard cap). More than 3 topics typically produces a path that weaves them together across weeks rather than covering each separately.

Step 3 — How Do You Learn Best? (Learning Style)

Difficulty level:

- `foundational` — Build from first principles; assumes no prior background
- `intermediate` — Deepen and apply knowledge; assumes general familiarity
- `advanced` — Leading-edge strategy and nuance; assumes expertise

Format preference:

- `readings` — Long-form analytical content
- `case_studies` — Real-world transformation narratives
- `interactive` — Quiz and reflection heavy
- `mixed` — Balanced across all formats (recommended default)

Step 4 — Plan Your Schedule

Time per week: 30 min, 1 hr, 2 hrs, 4 hrs, 8 hrs **Timeline:** 1 week, 2 weeks, 1 month (4 weeks), 2 months (8 weeks), 3 months (12 weeks)

The total learning hours (time × weeks) is calculated and displayed in real time. The backend uses this to determine how many items to generate per week:

```
items_per_week = max(2, min(6, floor(time_per_week_minutes / 28)))
```

A learner with 2 hrs/week gets ~4 items/week. A learner with 30 min/week gets 2.

Step 5 — Your Goals

Select 1 or more learning goals from 8 options. Optionally add a free-text custom goal (500 character max).

Goals directly influence what the AI emphasizes. A learner studying health economics with goal **financial-analysis** ("Conduct healthcare financial and economic analysis") will receive content heavy in benchmark calculations, ROI frameworks, and payer contract mechanics. The same learner with goal **strategic-leadership** will receive content emphasizing organizational decision-making and transformation strategy.

2.3 Path Generation

After completing the wizard, click **Generate My Learning Path**. Generation typically takes **15–45 seconds** depending on path length.

During generation the UI shows a loading animation. The 90-second timeout on the Next.js proxy will surface an error for unusually long generations.

What happens during generation (summary):

- 1. The backend fetches the live Sanity CMS content catalog
 - 2. The catalog is embedded into the AI prompt
 - 3. The Groq LLM generates the full curriculum in JSON
 - 4. Each item's **platform_link** is validated against the real catalog
 - 5. Personalized bridge paragraphs are generated for matched case studies
 - 6. The complete path is returned and stored in your browser's localStorage
-

2.4 Working Through Your Path

Once generated, the path viewer shows:

- **Path header** — title, description, total hours, difficulty badge, key skills
- **Weekly accordion** — each week has a theme and 2–6 items
- **Item cards** — collapsed by default; click to expand

Expanded item card contains:

- Type badge (Reading / Case Study / Knowledge Check)
- Estimated time
- Title and description
- **Content block** — the AI-generated lesson text (100–130 words)
- **Key Concepts** — 2–4 domain terms as pill tags
- **Reflection question** — a prompt to deepen thinking
- **"Why this case study matters for your path"** (*case studies with confirmed Academy match only*) — a personalized 2–3 sentence bridge paragraph explaining the specific relevance to where you are in your curriculum
- **Read in HTR Academy** button (*only shown for confirmed specific document matches*) — links to the actual published module or case study
- **Knowledge Check quiz** (*knowledge_check items*) — 3 questions; select answers and submit to see explanations; answers are saved
- **Notes** — free-text field saved per item
- **Mark as Complete** button

Progress is tracked as a percentage across all items and displayed on both the path card (in the path list) and the weekly header.

2.5 Managing Multiple Paths

You can create multiple paths. The path list view shows all saved paths with:

- Title and description

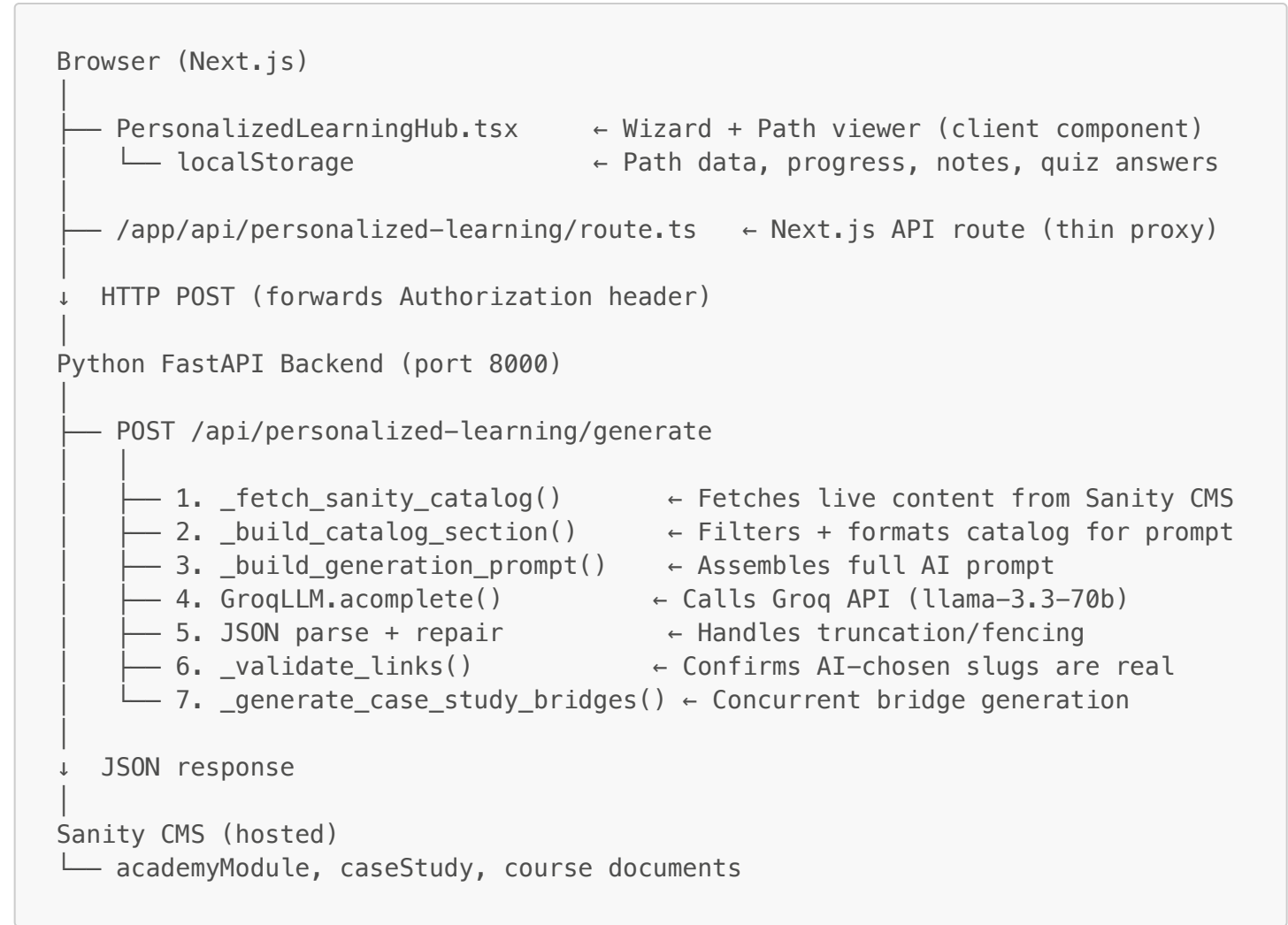
- Creation date and last accessed date
- Progress bar and percentage
- **Continue / View / Delete** actions

There is no hard limit on the number of saved paths (browser storage permitting).

2.6 Progress Persistence

All progress, quiz answers, and notes are saved automatically to `localStorage` on every interaction. They persist across page refreshes and browser restarts. They are **device-specific** — progress on your laptop does not sync to your phone.

3. Architecture Overview



4. Data Flow — End to End

4.1 Request Phase

The browser sends a `POST` to `/api/personalized-learning` (Next.js route) with:

```
{
  "role": "administrator",
  "experience_years": "6-10",
  "topics": ["health-economics", "payment-reform", "leadership"],
  "difficulty": "intermediate",
  "format_preference": "mixed",
  "time_per_week_hours": 2,
  "timeline_weeks": 4,
  "goals": ["vbc-implementation", "financial-analysis"],
  "custom_goal": null
}
```

The Next.js proxy forwards this with the user's `Authorization: Bearer <token>` header to `http://localhost:8000/api/personalized-learning/generate` with a 90-second timeout.

4.2 Backend Processing

Step 1 — Input validation Pydantic validates and sanitizes all fields:

- `topics` truncated to 8 max
- `timeline_weeks` clamped to 1–12
- `time_per_week_hours` clamped to 0.25–20.0
- `custom_goal` stripped and truncated to 500 chars

Step 2 — Sanity catalog fetch The backend calls Sanity's CDN API to fetch all published `academyModule`, `caseStudy`, and `course` documents with their slugs, titles, summaries, learning objectives, and pillars. The response is cached only for the duration of this request (no server-level caching currently). If Sanity is unreachable, an empty catalog is returned and the feature degrades gracefully — paths generate without Academy links.

Step 3 — Prompt construction The catalog is filtered to documents whose `pillar` matches the user's topic pillars (up to 8 modules and 5 case studies prioritized by relevance), then formatted as a plaintext list:

```
/academy/modules/intro-value-based-care-volume-to-value → Introduction to Value-Based Care: From Volume to Value
/academy/case-studies/vermont-all-payer-aco-case-study → Vermont: Building a Statewide All-Payer ACO from the Ground Up
```

This list is embedded directly in the AI prompt under the heading `HTR CONTENT LIBRARY`. The AI is instructed to set `platform_link` on each generated item to the best-matching URL from this list, or `null` if no strong fit exists.

Step 4 — LLM generation The full prompt (typically 1,500–2,500 tokens depending on path length and catalog size) is sent to Groq's `llama-3.3-70b-versatile` model with:

- `temperature=0.65` — creative enough for varied content, stable enough for consistent JSON
- `max_tokens=8000` — leaves headroom under Groq's 8,192 output limit to prevent JSON truncation

The model returns a JSON object conforming to the schema embedded in the prompt.

Step 5 — JSON repair The response is stripped of markdown fences if present. The parser attempts `json.loads()` on the full output; if that fails (due to truncation), it walks backward from the end of the string looking for the last valid closing brace, effectively repairing truncated responses.

Step 6 — Link validation `_validate_links()` iterates over every item's `platform_link`:

- Links containing `/academy/modules/<slug>` are checked against `valid_module_slugs` (the set of real slugs from the catalog)
- Links containing `/academy/case-studies/<slug>` are checked against `valid_case_study_slugs`
- Any other value (hallucinated slug, generic tab URL, empty string) is set to `null`
- `knowledge_check` items always receive `null`

This is the anti-hallucination gate. The AI can only produce links to documents that existed in the catalog at generation time.

Step 7 — Case study bridge generation For each `case_study` item with a validated `platform_link`, a separate Groq call is made (all concurrent via `asyncio.gather`) to generate a personalized 2–3 sentence `relevance_bridge`. This call uses:

- `temperature=0.25` — factual, specific prose (lower creativity than the curriculum generation)
- The learner's role, topics, goals, current week theme, and the item's title as context
- The matched case study's title and summary

- Strict prompt rules forbidding generic openers like "This case study..." — must start with the organization name or a concrete fact

Bridge generation failures per item are non-fatal (caught and logged); the link still renders without the bridge.

4.3 Response Schema

```
{
  "success": true,
  "path": {
    "title": "...",
    "description": "...",
    "estimated_weeks": 4,
    "total_hours": 8.0,
    "difficulty_level": "intermediate",
    "key_skills": ["skill1", "skill2", "skill3", "skill4", "skill5"],
    "weeks": [
      {
        "week": 1,
        "theme": "...",
        "focus_areas": ["area1", "area2"],
        "items": [
          {
            "id": "w1-i1",
            "type": "reading",
            "title": "...",
            "description": "...",
            "content": "100-130 word lesson...",
            "platform_link": "/academy/modules/intro-value-based-care-volume-
to-value",
            "relevance_bridge": null,
            "estimated_minutes": 15,
            "key_concepts": ["concept1", "concept2"],
            "reflection_question": "...",
            "questions": null
          },
          {
            "id": "w1-i2",
            "type": "case_study",
            "title": "...",
            "description": "...",
            "content": "100-130 word case narrative...",
            "platform_link": "/academy/case-studies/vermont-all-payer-aco-case-
study",
            "relevance_bridge": "Vermont's All-Payer ACO Model – launched in
2018 under the Green Mountain Care Board – is the direct real-world
implementation of the shared savings mechanics you just studied. As a hospital
administrator at an organization evaluating ACO entry, the GMCB's approach to
multi-payer alignment and total cost of care benchmarks maps precisely to the
contract decision framework you are building.",
            "estimated_minutes": 15,
            "key_concepts": ["concept1"],
            "reflection_question": "...",
            "questions": null
          }
        ]
      }
    ],
    "recommended_resources": [...]
  },
  "preferences": { ... }
}
```


Note: `relevance_bridge` is only populated on `case_study` items with a confirmed `platform_link`. All other items have `relevance_bridge: null`.

5. Backend — Deep Technical Reference

5.1 File Location

```
backend/routers/personalized_learning.py
```

Registered in `backend/main.py` as `personalized_learning_router`.

5.2 Key Functions Reference

`_fetch_sanity_catalog() → List[Dict]`

Fetches all published `academyModule`, `caseStudy`, and `course` documents from Sanity CMS. Returns empty list if `SANITY_PROJECT_ID` or `SANITY_API_TOKEN` are not set.

Timeout: 15 seconds (httpx) **On failure:** Logs a warning, returns `[]`. The endpoint continues and generates paths without Academy links. **Fields fetched:** `_type`, `title`, `slug`, `pillar`, `summary`, `learningObjectives`

`_build_catalog_section(catalog, topics) → str`

Filters the catalog by the user's topic pillars and formats it as a plaintext block for injection into the AI prompt.

Pillar filter logic:

```
topic_pillars = {TOPIC_TO_PILLAR.get(t) for t in topics} - {None}
# Documents with pillar=="All" are always included
```

Caps: 8 modules, 5 case studies (to stay within token budget). **If catalog is empty:** Returns `""` — the prompt section is omitted entirely.

`_build_generation_prompt(prefs, catalog) → str`

Assembles the complete generation prompt. Key sections:

- LEARNER PROFILE** — role, experience, difficulty, time budget, topics, goals
- HTR PLATFORM CONTEXT** — the 5 content pillars and Vermont-specific context
- HTR CONTENT LIBRARY** — the real published documents (from `_build_catalog_section`)
- GENERATION INSTRUCTIONS** — rules including the `platform_link` instruction
- JSON schema** — a complete example of the expected output structure

ROLE_FRAMING dictionary maps each role ID to a short framing phrase used in the learner profile section:

```
"administrator": "operational efficiency, workforce, and organizational strategy"
"clinician":      "front-line clinical care delivery and quality outcomes"
# etc.
```

items_per_week calculation:


```
mins_per_week = prefs.time_per_week_hours * 60
items_per_week = max(2, min(6, int(mins_per_week / 28)))
```

Target: approximately 28 minutes per item.

`_validate_links(path_data, catalog) → Dict`

Post-generation anti-hallucination gate. Builds sets of valid slugs from the catalog and checks every `platform_link` the AI generated.

Rules:

- `/academy/modules/<slug>` → slug must be in `valid_module_slugs`
- `/academy/case-studies/<slug>` → slug must be in `valid_case_study_slugs`
- Any other value → `None`
- `knowledge_check` items → always `None`

Log output: Warnings for nulled hallucinated slugs; debug for each validated link.

`_generate_case_study_bridges(path_data, catalog, prefs) → Dict`

Orchestrates concurrent bridge generation for all matched case study items.

```
await asyncio.gather(*tasks)
```

All bridges for a path (typically 1–4) are generated simultaneously, adding minimal additional latency beyond the slowest single call (~2–5 seconds).

`_generate_single_bridge(item, doc, prefs, week_theme, llm) → None`

Generates one bridge paragraph in-place on `item["relevance_bridge"]`.

Prompt inputs used:

- `prefs.role` (human-readable)
- Up to 4 topics (human-readable labels)
- Up to 3 goals (human-readable labels)
- `week_theme` from the current week
- `item["title"]` — what the learner just read
- `doc["title"]` and `doc["summary"]` — the matched case study

Temperature: 0.25 (factual, specific) **Failure handling:** `try/except` per item — never raises, only logs

5.3 LearningPreferences — Validation Rules

Field	Type	Validator
<code>role</code>	<code>str</code>	No constraint (trusts UI)
<code>experience_years</code>	<code>str</code>	No constraint (trusts UI values: "0-2", "3-5", "6-10", "10+")
<code>topics</code>	<code>List[str]</code>	Min 1; truncated to 8
<code>difficulty</code>	<code>str</code>	No constraint (trusts UI)
<code>format_preference</code>	<code>str</code>	No constraint
<code>time_per_week_hours</code>	<code>float</code>	Clamped: 0.25 – 20.0

Field	Type	Validator
timeline_weeks	int	Clamped: 1 – 12
goals	List[str]	No constraint
custom_goal	Optional[str]	Stripped and truncated to 500 chars

5.4 Domain Lookup Tables

TOPIC_TO_PILLAR — Maps topic IDs to Sanity pillar values for catalog pre-filtering. `leadership` maps to `None` (no single pillar; the full catalog is used).

TOPIC_TO_TRACK — Maps topic IDs to the published learning track IDs in `learning-tracks-data.ts`. Used by the frontend link resolver for legacy URL upgrades. Must stay in sync with the track IDs defined in the data file.

6. Frontend — Deep Technical Reference

6.1 File Locations

File	Purpose
frontend/components/academy/PersonalizedLearningHub.tsx	Main component — wizard + viewer (1,646 lines)
frontend/app/api/personalized-learning/route.ts	Next.js proxy to Python backend
frontend/app/academy/page.tsx	Academy hub — registers the Personalized Learning tab
frontend/lib/data/learning-tracks-data.ts	Static learning track definitions
frontend/components/academy/LearningTracksHub.tsx	Learning tracks viewer (reads <code>?track=</code> param)

6.2 localStorage Schema

Storage key: `htr_personalized_paths_v2`

```
interface SavedPath {
  id: string; // UUID generated at creation
  created_at: string; // ISO timestamp
  last_accessed: string; // ISO timestamp, updated on open
  preferences: LearningPreferences; // The wizard inputs used to generate
this path
  path: GeneratedPath; // The full AI-generated curriculum
  progress: Record<string, boolean>; // itemId → completed (true/false)
  quiz_answers: Record<string, number[]>; // itemId → array of selected answer
indices
  notes: Record<string, string>; // itemId → free text note
}
```

Storage key versioning: The key is `htr_personalized_paths_v2`. If a breaking schema change is made, bump the version (e.g., `v3`). Old data under the previous key is simply abandoned — there is no migration logic currently. See [Section 12.2](#) for the upgrade path.

LoadPaths() — reads and parses localStorage; returns `[]` on any error **savePaths(paths)** — serializes the full array back to localStorage on every change **calcProgress(path)** — `completedItems / totalItems * 100` **nextUncompletedItem(path)** — returns first incomplete item index for resume behavior

6.3 Key Component: PathItemCard

The most complex rendering component. Accepts:

```
{
  item: PathItem;
  topics: string[];           // from the path's preferences – used by
  resolveAcademyLink
  isCompleted: boolean;
  quizAnswers: number[];
  note: string;
  onToggleComplete: () => void;
  onQuizAnswer: (answers: number[]) => void;
  onNoteChange: (note: string) => void;
}
```

Link rendering logic (critical business rule):

```
// ONLY render the outbound link section when platform_link points to
// a specific real document – never for null, tab URLs, or course URLs
{item.platform_link &&
  (item.platform_link.includes("/academy/modules/") ||
  item.platform_link.includes("/academy/case-studies/")) && (
<div className="space-y-3">
  {/* Bridge callout – only for case_study items with relevance_bridge */}
  {item.type === "case_study" && item.relevance_bridge && (
    <div className="bg-indigo-50 border border-indigo-200 rounded-xl p-4">
      ...
    </div>
  )}
  {/* Link button */}
  <a href={resolveAcademyLink(item, topics).href}>
    {resolveAcademyLink(item, topics).label}
  </a>
</div>
)}
```

This is the enforcement point for the product rule: **if we cannot confirm the destination is a specific, real, relevant document, we show nothing rather than a generic link.**

6.4 resolveAcademyLink(item, topics)

Maps a platform_link URL to a human-readable label. Uses the topics array to upgrade legacy localStorage paths that have a generic /academy?tab=tracks URL without a track= param.

```
// Legacy upgrade path: generic tab URL → best track for user's topics
if (link.includes("tab=tracks")) {
  const trackMatch = link.match(/(?:&)track=([^\&]+)/);
  const trackId = trackMatch ? trackMatch[1] : bestTrackForTopics(topics);
  const href = trackId && !trackMatch
    ? `/academy?tab=tracks&track=${trackId}`
    : link;
  return { href, label: name ? `Explore the ${name}` : "Explore Related Learning Tracks" };
}
```

In practice, for all paths generated after the current architecture, platform_link will only ever be a specific document URL or null, so the tab URL branch is only reached for paths generated before the current system was

deployed.

6.5 Learning Tracks Integration

The Learning Tracks tab (`LearningTracksHub.tsx`) reads the `?track=` URL parameter:

```
const searchParams = useSearchParams();
const highlightedTrackId = searchParams.get("track");
```

When a user navigates from a personalized path item to `/academy?tab=tracks&track=value-based-care-track`, the matching `TrackCard`:

- Initializes with `expanded=true` (module list open)
- Fires `scrollIntoView({ behavior: "smooth", block: "start" })` on mount
- Renders with an indigo ring border to visually distinguish it

This requires `LearningTracksHub` to be a `"use client"` component (it is).

7. The Sanity CMS Content Library

7.1 Current Published Content (as of March 2026)

Academy Modules (18):

Slug	Title	Pillar
<code>vbc-fundamentals-m1</code>	Why the Way We Pay for Healthcare Is Breaking the System	All
<code>intro-value-based-care-volume-to-value</code>	Introduction to Value-Based Care: From Volume to Value	Economics
<code>vbc-fundamentals-module-2-policy-pillar</code>	VBC Fundamentals — Module 2: The Policy Pillar	Policy
<code>vbc-fundamentals-module-3-economics-pillar</code>	VBC Fundamentals — Module 3: The Economics Pillar	Economics
<code>vbc-fundamentals-module-4-technology-pillar</code>	VBC Fundamentals — Module 4: The Technology Pillar	Technology
<code>vbc-clinical-m5</code>	From Volume to Value at the Bedside	Clinical
<code>vbc-equity-m6</code>	Who Gets Left Behind: Health Equity in VBC	Equity
<code>understanding-risk-contracts-and-shared-savings</code>	Understanding Risk Contracts and Shared Savings Models	Economics
<code>care-management-vbc-strategies</code>	Care Management Strategies for VBC Success	Economics
<code>precision-medicine-m1</code>	From One-Size-Fits-All to You: The Precision Medicine Revolution	All
<code>precision-medicine-m2-policy</code>	Who Writes the Rules: Regulatory Framework for Precision Medicine	Policy
<code>precision-medicine-m3-economics</code>	The Billion-Dollar Molecule: Economics of Precision Medicine	Economics
<code>precision-medicine-m4-technology</code>	The Genomic Stack: Sequencing, AI, and Digital Infrastructure	Technology

Slug	Title	Pillar
precision-medicine-m5-clinical	From Mutation to Medicine: Precision Medicine in Clinical Practice	Clinical
precision-medicine-m6-equity	The Diversity Deficit: Why Precision Medicine Must Work for Everyone	Equity
defining-health-equity-concepts-frameworks-measurement	Defining Health Equity: Concepts, Frameworks, and Measurement	Equity
social-determinants-of-health-clinical-integration	Social Determinants of Health: Clinical Integration and Community Partnerships	Equity
building-health-equity-programs-roi-measurement	Building Health Equity Programs: ROI, Accountability, and Organizational Change	Equity

Case Studies (7):

Slug	Title	Pillar
vermont-all-payer-aco-case-study	Vermont: Building a Statewide All-Payer ACO from the Ground Up	Economics
saving-mercy-regional	Saving Mercy Regional	Economics
camden-coalition-hotspotting-complex-patients	Camden Coalition: The Original Hotspotting Model for High-Need Patients	Equity
california-cmqcc-maternal-mortality-bundles	California CMQCC: Reducing Maternal Mortality Through Clinical Bundles	Clinical
intermountain-hospital-at-home-operations	Intermountain Health: Hospital-at-Home Program Design and Operations	Clinical
kaiser-permanente-ai-sepsis-detection	Kaiser Permanente: AI-Augmented Sepsis Detection at Scale	Technology
ai-in-ed	AI in the ED	Technology

Courses (2):

Slug	Title
value-based-care-fundamentals	Value-Based Care Fundamentals
precision-medicine-fundamentals	Precision Medicine Fundamentals

7.2 The Three Published Learning Tracks

Defined in `frontend/lib/data/learning-tracks-data.ts`. **Every slug in this file must match a real `Sanity academyModule` document.** The system has no runtime validation for this — mismatches result in 404 pages for users who click module links.

Track ID	Title	Module Count	Total Time
value-based-care-track	Value-Based Care: Foundation to Advanced	9	~380 min
precision-medicine-track	Precision Medicine: From Revolution to Bedside	6	~255 min
health-equity-track	Health Equity in Practice	5	~205 min

7.3 Adding New Content — Operational Checklist

When publishing a new Sanity document that should be linked from personalized learning paths:

1. **Publish the document in Sanity** with a `slug.current`, `title`, `summary`, `pillar`, and `learningObjectives` fields populated. The `summary` and `learningObjectives` are particularly important — the AI uses the title and these fields when deciding which document best matches a curriculum item.

2. **Verify the Sanity query returns it:**
- ```
From backend directory
python3 -c "
import os, json, urllib.request, urllib.parse
(load .env and run the catalog query)
"
```
3. **If it belongs in a Learning Track**, add it to `learning-tracks-data.ts` in the correct position with accurate module metadata. The `slug` field must exactly match the Sanity slug.

4. **Update `TOPIC_TO_PILLAR`** in `personalized_learning.py` if the new content introduces a new topic area not currently mapped.

5. **No code restart is required** for new Sanity content — the catalog is fetched live on every generation request.

## 8. Persistence Layer — localStorage

### 8.1 Why localStorage (Current)

Chosen for the initial implementation because:

- Zero infrastructure cost — no database table, no migration, no API
- Instant read/write — no latency on progress save
- Works offline after initial page load

### 8.2 Limitations

- **Device-specific** — no cross-device sync
- **Quota limits** — browsers impose ~5–10 MB per origin; a user with many long paths could hit this limit (large JSON blobs)
- **No backup** — clearing browser data permanently destroys all paths and progress
- **No server-side analytics** — we cannot see which items users are completing, which paths are generating the most engagement, or which case studies are driving the most Academy traffic

### 8.3 Storage Key Versioning

Current key: `htr_personalized_paths_v2`

When a breaking schema change is needed (see [Section 12.2](#)), increment the key. A migration function should be added to `loadPaths()` to read the old key and convert data to the new schema before writing it to the new key.

## 9. Authentication & Access Control

### 9.1 Subscriber Gate

The endpoint is protected by `require_subscriber` from `services/auth.py`:

```
@router.post("/api/personalized-learning/generate")
async def generate_learning_path(
 prefs: LearningPreferences,
```

```
 user: AuthedUser = Depends(require_subscriber),
):
```

This dependency:

- 1. Reads the `Authorization: Bearer <token>` header
- 2. Validates the JWT against Supabase
- 3. Checks that the user's profile has an active subscription tier
- 4. Returns `403 Forbidden` for non-subscribers or expired subscribers

## 9.2 Token Forwarding

The Next.js proxy (`route.ts`) forwards the browser's `Authorization` header verbatim to the Python backend:

```
const authHeader = req.headers.get("Authorization") || "Bearer dev";
```

Note the `|| "Bearer dev"` fallback — in local development, if no auth header is present, `Bearer dev` is sent. The Python backend must handle this gracefully in dev mode (it does via the `dev` token bypass in the auth service).

## 9.3 What Subscribers Can Do That Free Users Cannot

- Generate learning paths (the primary gate)
- Access the path viewer (the viewer itself checks for `require_subscriber` indirectly — the path list will be empty for free users since they cannot generate)

**The wizard is visible to all users.** This is intentional — free users can explore the wizard to see what they would get, then hit the gate at generation time. This is a conversion mechanism: seeing the wizard builds intent before the paywall.

# 10. Environment Variables & Configuration

All variables are set in `backend/.env`. Required variables for this feature:

| Variable                        | Required                               | Purpose                                                     |
|---------------------------------|----------------------------------------|-------------------------------------------------------------|
| <code>GROQ_API_KEY</code>       | Yes                                    | Authenticates calls to Groq's LLM API                       |
| <code>MODEL_SUBSCRIBER</code>   | Yes                                    | The model ID used for path generation and bridge generation |
| <code>SANITY_PROJECT_ID</code>  | Yes*                                   | Sanity project identifier                                   |
| <code>SANITY_API_TOKEN</code>   | Yes*                                   | Sanity API read token                                       |
| <code>SANITY_DATASET</code>     | No (default: <code>production</code> ) | Sanity dataset name                                         |
| <code>SANITY_API_VERSION</code> | No (default: <code>2023-10-01</code> ) | Sanity API version                                          |

\*If `SANITY_PROJECT_ID` or `SANITY_API_TOKEN` are absent, the feature degrades gracefully: paths generate but contain no Academy links and no bridge paragraphs.

Frontend variables:

| Variable                        | Required                                          | Purpose                   |
|---------------------------------|---------------------------------------------------|---------------------------|
| <code>PYTHON_BACKEND_URL</code> | No (default: <code>http://localhost:8000</code> ) | URL of the Python backend |

Set in `frontend/.env.local` for local development and in the deployment environment for production.



## 11. Maintenance & Operations

### 11.1 Monitoring — What to Watch

**Backend logs** (via `log = logging.getLogger("htr-brain")`):

| Log message                                                  | Meaning                          | Action                                                  |
|--------------------------------------------------------------|----------------------------------|---------------------------------------------------------|
| Sanity catalog: N documents fetched                          | Healthy catalog fetch            | None                                                    |
| Sanity credentials not set – platform link matching disabled | Env vars missing                 | Add to .env                                             |
| Sanity catalog fetch failed                                  | Sanity unreachable or auth error | Check Sanity token; check network                       |
| AI hallucinated module slug 'X' – nulling link               | AI invented a non-existent slug  | Normal and expected occasionally; log frequency matters |
| Generated N case study bridge(s)                             | Bridge generation succeeded      | None                                                    |
| Bridge generation failed for 'X'                             | One bridge failed (non-fatal)    | Check Groq rate limits                                  |
| LLM returned malformed JSON that could not be repaired       | Full generation failure          | Check Groq status; consider retry                       |

**Key metrics to track:**

- Generation success rate (successful responses / total requests)
- Average generation latency (Groq call + bridge calls)
- Bridge generation success rate
- Hallucination rate (how often are links nulled by `_validate_links`)
- Sanity catalog fetch latency

### 11.2 Groq Model Management

The model is referenced via `MODEL_SUBSCRIBER` in `config.py`. Current model: `llama-3.3-70b-versatile`.

**To change the model:**

1. Update `MODEL_SUBSCRIBER` in `.env`
2. Restart the backend
3. **Test thoroughly:** different models produce different JSON reliability, different verbosity in `content` fields, and different instruction-following fidelity for the `platform_link` rule

**If Groq changes model availability** (deprecates a model):

- Generation requests will return 500 errors
- Check Groq's model availability at `console.groq.com`
- Update `MODEL_SUBSCRIBER` to the new model ID

### 11.3 Temperature Settings

Two temperatures are used in this feature:

| Call                                             | Temperature | Rationale                                                                           |
|--------------------------------------------------|-------------|-------------------------------------------------------------------------------------|
| Curriculum generation ( <code>acomplete</code> ) | 0.65        | Needs creative variety across users and topics; too low produces repetitive content |

| Call                                                          | Temperature | Rationale                                                                                    |
|---------------------------------------------------------------|-------------|----------------------------------------------------------------------------------------------|
| Bridge generation<br>( <code>_generate_single_bridge</code> ) | 0.25        | Needs factual, specific, grounded prose; higher temperatures produce generic or flowery text |

When to adjust generation temperature:

- **Lower (0.5–0.55):** If the AI is producing inconsistent JSON structure or hallucinating slugs frequently. Lower temperature increases adherence to the JSON schema and the `platform_link` instruction.
- **Raise (0.7–0.75):** If content feedback indicates paths feel repetitive or formulaic across users with similar inputs.

When to adjust bridge temperature:

- **Lower (0.1–0.2):** If bridges are producing inaccurate or speculative claims about case study content.
- **Raise (0.3–0.4):** If bridges feel mechanical or overly similar across different cases. Rarely needed.

## 11.4 Routine Maintenance Tasks

Monthly:

- Check Sanity catalog completeness — run the catalog query manually and confirm all expected documents are present
- Review backend logs for hallucination rate — if `AI hallucinated module slug` warnings are frequent, it indicates the model is struggling with the content library section of the prompt
- Review Groq usage dashboard for token consumption trends

When new Sanity content is published:

- Follow the checklist in [Section 7.3](#)
- No restart required — the catalog is fetched live

When a subscriber reports missing links or missing bridge text:

1. Ask them to regenerate their path (older paths were generated under previous code)
2. Check backend logs for hallucination warnings during their generation
3. Verify the Sanity document they expected is published and has `summary` and `learningObjectives` populated

## 12. Scaling & Upgrade Paths

### 12.1 Supabase Persistence (High Priority)

**Current state:** All path data lives in the user's browser `localStorage`. **Problem:** No cross-device sync, no analytics, no backup, potential quota limits.

Recommended upgrade:

Create two Supabase tables:

```
-- Stores the full generated path and preferences
CREATE TABLE personalized_learning_paths (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
 title TEXT NOT NULL,
 description TEXT,
 preferences JSONB NOT NULL, -- LearningPreferences object
 path_data JSONB NOT NULL, -- Full GeneratedPath object
 created_at TIMESTAMPTZ DEFAULT NOW(),
 last_accessed TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Stores per-item progress, quiz answers, and notes
CREATE TABLE personalized_path_progress (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 path_id UUID NOT NULL REFERENCES personalized_learning_paths(id) ON
DELETE CASCADE,
 user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
 item_id TEXT NOT NULL, -- matches PathItem.id
 completed BOOLEAN DEFAULT FALSE,
 quiz_answers INTEGER[], -- array of selected answer indices
 note TEXT,
 updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX ON personalized_learning_paths(user_id);
CREATE INDEX ON personalized_path_progress(path_id, item_id);
```

Frontend migration:

- Replace `loadPaths()` / `savePaths()` with Supabase client calls
- Keep `localStorage` as a write-through cache for offline support
- Migrate existing `localStorage` data to Supabase on first sign-in after deployment

**Backend impact:** None — paths are still returned from the `generate` endpoint and stored by the frontend.

12.2 Migrating `localStorage` Schema

If a breaking change to `SavedPath` structure is needed:

```
// In loadPaths():
function loadPaths(): SavedPath[] {
 const raw = localStorage.getItem("htr_personalized_paths_v2");
 const legacyRaw = localStorage.getItem("htr_personalized_paths_v1");

 if (!raw && legacyRaw) {
 // Migrate v1 → v2
 const legacyPaths = JSON.parse(legacyRaw);
 const migrated = legacyPaths.map(migratePath);
 savePaths(migrated);
 localStorage.removeItem("htr_personalized_paths_v1");
 return migrated;
 }
 return raw ? JSON.parse(raw) : [];
}
```

12.3 Sanity Catalog Caching

**Current state:** Catalog is fetched live on every generation request (~200–800ms). **At scale:** With many concurrent users, this creates N simultaneous Sanity API calls.

**Recommended upgrade — in-process cache:**

```
import time
from typing import Optional

_catalog_cache: Optional[List[Dict]] = None
_catalog_cache_time: float = 0
_CATALOG_TTL_SECONDS = 300 # 5 minutes

async def _fetch_sanity_catalog() -> List[Dict]:
 global _catalog_cache, _catalog_cache_time
```

```
 now = time.time()
 if _catalog_cache is not None and (now - _catalog_cache_time) <
_CATALOG_TTL_SECONDS:
 return _catalog_cache
 # ... existing fetch logic ...
 _catalog_cache = results
 _catalog_cache_time = now
 return results
```

This reduces Sanity API calls from N per second to 1 per 5 minutes at any traffic level. Invalidate by restarting the backend or setting TTL to 0 temporarily.

For multi-process deployments (multiple uvicorn workers), use Redis instead of in-process state.

## 12.4 Adding New Topics

### Steps:

1. Add the new topic ID and label to **TOPICS** array in **PersonalizedLearningHub.tsx**
2. Add the topic ID → pillar mapping to **TOPIC\_T0\_PILLAR** in **personalized\_learning.py**
3. Add the topic ID → track mapping to **TOPIC\_T0\_TRACK** in both **personalized\_learning.py** and **PersonalizedLearningHub.tsx**
4. Publish relevant Sanity content with the corresponding pillar value
5. If a new Learning Track is warranted, add it to **learning-tracks-data.ts** with real Sanity slugs

**Limit consideration:** The UI enforces a max of 5 selected topics per path. The backend accepts up to 8 (validator: **v[:8]**). The **\_build\_catalog\_section** cap of 8 modules + 5 case studies means the prompt stays manageable even if the user selects multiple topics that span many pillars.

## 12.5 Adding New Roles

### Steps:

1. Add the new role to **ROLES** array in **PersonalizedLearningHub.tsx**
2. Add the role ID → framing phrase to **ROLE\_FRAMING** in **personalized\_learning.py**

The framing phrase is injected directly into the AI prompt as "focus on [framing]" and has a significant effect on content tone and emphasis. Write it carefully — it should describe the role's primary professional concerns in 6–10 words.

## 12.6 Increasing Path Length Beyond 12 Weeks

The current cap is 12 weeks (backend validator: **max(1, min(12, v))**).

To increase it:

1. Update the validator: **max(1, min(N, v))**
2. Update the UI **TIMELINE\_OPTIONS** to add the new option
3. **Test token consumption** — a 16-week path at 4 items/week is ~64 items of JSON, which may approach the 8,000 token output limit. Consider reducing **content** word count or items per week for longer paths, or upgrading to a model with a larger output window.

## 12.7 Moving to a More Capable Model

If Groq makes a more capable model available (e.g., a 405b parameter model), or if you want to test OpenAI GPT-4o or Claude as the generation model:

1. **For Groq models:** Update **MODEL\_SUBSCRIBER** in **.env**

2. **For non-Groq models:** Replace `GroqLLM` import and instantiation with the appropriate LlamaIndex LLM class (e.g., `from llama_index.llms.openai import OpenAI`)
3. **Test temperature calibration** — different models have different temperature scales; the `0.65 / 0.25` settings were tuned for llama-3.3-70b
4. **Test JSON reliability** — some models require stronger schema enforcement; consider adding `response_format={"type": "json_object"}` if the model supports it
5. **Test platform\_link instruction following** — the core business rule that prevents generic links depends on the model faithfully following the "use ONLY these URLs" instruction. Verify with multiple generations.
- 

## 13. Troubleshooting Guide

### 13.1 "The AI returned malformed content. Please try again."

**Cause:** The LLM response could not be parsed as valid JSON even after the repair attempt. **Common triggers:** Groq API intermittent errors, network timeout during streaming, unusually large path (many weeks) hitting the token limit. **Fix:** Retry. If persistent, check Groq status page. If the error is specifically for long paths (8+ weeks), the model may be hitting the 8,000 token output limit — reduce `max_tokens` or the path length.

### 13.2 "Cannot reach the AI backend."

**Cause:** The Next.js proxy cannot connect to the Python backend at the configured URL. **Fix:** Ensure the backend is running: `cd backend && uvicorn main:app --reload --port 8000`. Check `PYTHON_BACKEND_URL` in the frontend environment.

### 13.3 No Academy Links Appear in the Generated Path

**Most likely cause:** The Sanity catalog fetch is failing, so the AI prompt contains no content library section, the AI sets all `platform_link` values to `null`, and `_validate_links` confirms them as null.

#### Diagnosis:

1. Check backend logs for `Sanity catalog fetch failed`
2. Check `SANITY_PROJECT_ID` and `SANITY_API_TOKEN` are set in `.env`
3. Test the Sanity connection directly:

```
python3 -c "
run _fetch_sanity_catalog() manually and print result length
"
```

**Secondary cause:** The AI set `platform_link` to non-null values but `_validate_links` nulled them all (slugs hallucinated). Check logs for `AI hallucinated module slug` warnings. If frequent, the catalog section is not being followed — try lowering generation temperature.

### 13.4 "Why This Case Study Matters" Bridge is Missing

#### Possible causes:

1. The case study item has no `platform_link` — it did not match a real Sanity case study. Bridge generation only runs for validated links.
2. Bridge generation failed — check logs for `Bridge generation failed for` warnings.
3. The user is viewing an older path generated before bridge generation was implemented — regenerate.

### 13.5 A Generated Link Leads to a 404 Page

**Cause:** The AI generated a valid-format link but to a Sanity document that is not published or has been deleted since the path was generated.

`_validate_links` checks against the live catalog at generation time. If a document is later unpublished, existing paths pointing to it will 404.

Fix:

- For individual users: regenerate the path
- Long-term: implement a link health-check sweep that periodically fetches the catalog and nulls stale links in stored paths

### 13.6 Learning Track Does Not Auto-Scroll/Expand When Navigating from a Path

**Cause:** The `?track=` URL parameter is not being read by `LearningTracksHub`.

Check:

1. The URL should be `/academy?tab=tracks&track=value-based-care-track` (or similar)
2. `LearningTracksHub` must be a `"use client"` component (it is)
3. `useSearchParams()` requires `Suspense` wrapping in Next.js 13+ App Router — if this breaks, wrap the component in `<Suspense>`

### 13.7 Path Generation is Very Slow (>45 seconds)

Causes:

- Groq API latency spike (check status page)
- Bridge generation for many case study matches (4+ concurrent calls)
- Large path size (12 weeks x 4 items = 48 items generates a large JSON blob)

Mitigation options:

- Add a server-side Sanity catalog cache to eliminate the catalog fetch from the critical path
- For very large paths, consider streaming the curriculum JSON and running bridge generation client-side after display

## 14. Known Limitations & Roadmap

### 14.1 Current Limitations

| Limitation                                                                      | Impact                                                          | Priority  |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------|
| localStorage only — no cross-device sync                                        | Subscribers lose progress switching devices                     | High      |
| No server-side analytics on path engagement                                     | Cannot measure which content drives retention                   | High      |
| Catalog fetched live on every request                                           | Adds ~200–800ms latency; N Sanity calls at scale                | Medium    |
| Bridge generation adds latency (concurrent but still serial with path display)  | Users wait for bridges before seeing path                       | Medium    |
| No path editing — must regenerate entirely to change preferences                | Friction for users who want to adjust difficulty or add a topic | Medium    |
| Module <code>estimatedMinutes</code> not in Sanity — static values in data file | Times shown in tracks are estimates, not from CMS               | Low       |
| Free users see the full wizard but hit a gate at generation                     | May cause frustration; also works as conversion mechanism       | By design |

### 14.2 Roadmap

**Phase 1 — Supabase persistence** (eliminates device-lock and enables analytics) **Phase 2 — Server-side catalog cache** (eliminates Sanity call from hot path) **Phase 3 — Path editing** (allow changing difficulty, adding topics without full regeneration) **Phase 4 — Admin analytics dashboard** (completion rates, popular paths, case study click-through rates) **Phase 5 — Adaptive paths** (path adjusts difficulty based on quiz scores over time)

---

## 15. Security Considerations

### 15.1 Input Validation

All user inputs are validated by Pydantic before touching the AI prompt. The `custom_goal` field (free text) is stripped and capped at 500 characters. This prevents prompt injection via the custom goal field from significantly affecting the output.

However: the AI is not a security boundary. Do not treat any AI-generated content as trusted input for other systems.

### 15.2 Slug Validation as Anti-Hallucination

`_validate_links` is not primarily a security measure — it's a correctness measure. The risk it guards against is the AI inventing URLs that do not exist on the platform. A hallucinated slug like `/academy/modules/made-up-article` would result in a 404 for the user. Validation ensures only real documents are linked.

### 15.3 Subscriber Token

The feature requires a valid subscriber JWT. The token is forwarded from the browser by the Next.js proxy and validated by the Python backend's `require_subscriber` dependency. The backend does not trust the client to self-report subscription status — it validates the token against Supabase on every request.

### 15.4 Sanity API Token

The `SANITY_API_TOKEN` is a read-only token used only to query published documents. It is never exposed to the browser and is only used server-side. Rotate it via the Sanity dashboard if it is compromised.

### 15.5 Rate Limiting

There is currently no per-user rate limiting on the generate endpoint. A single subscriber can make unlimited generation requests, each consuming significant Groq API tokens (curriculum generation ~1,500 tokens input / 2,000–6,000 tokens output; plus N bridge calls).

**Recommended:** Add a rate limiter at the FastAPI layer (e.g., `slowapi`) limiting generation to 5 requests per user per hour.

---